

Data-Driven Threshold Scheduling for LLM Inference: Preventing Memory Eviction Cascades

Ruicheng Ao¹, Gan Luo², David Simchi-Levi¹, Xinshang Wang³

¹Institute for Data, Systems, and Society, MIT

²School of Mathematical Sciences, Peking University

³Alibaba Group

aorc@mit.edu, luogan@stu.pku.edu.cn, dslevi@mit.edu, xinshang.w@alibaba-inc.com

Abstract

Large Language Models enable modern AI applications but incur high costs—ChatGPT’s daily inference exceeds \$700,000. We develop threshold-based scheduling algorithms for LLM inference under memory constraints. Dynamic Key-Value cache growth creates eviction risks that degrade throughput by 12-25% even when capacity is sufficient. Using fluid dynamics from queueing theory, we characterize equilibrium where arrivals match completions, revealing optimal memory M^* . Our algorithms—WAIT (known output lengths) and Nested WAIT (unknown outputs)—maintain systems near equilibrium through controlled admission. Nested WAIT uses on-the-fly classification: short prompts complete early while long prompts advance to later segments, eliminating output prediction. Validation on real-world LMSYS-Chat-1M data and NVIDIA A100 GPUs achieves 12-25% throughput improvement over vLLM and Sarathi.

Keywords

LLM inference, online scheduling, memory constraints, queueing theory, throughput optimization

1. Introduction

LLMs power chatbots, search, and programming assistants [1, 2], generating extensive operational data (arrival rates, prompt lengths, latency) that inform scheduling [3]. LLM inference has two phases: *prefill* (process input) and *decode* (generate output). The Key-Value cache stores representations but consumes GPU memory growing linearly with output length. Batching improves throughput but risks exceeding capacity C , forcing *eviction*—restarting prompts and wasting computation.

Eviction Challenge. Memory sufficiency doesn’t guarantee stability. Even when C equals optimal M^* , FCFS triggers cascading evictions. Example: 1-token prompts, $C = 12$, $\lambda = 4$. Equilibrium needs 4 prefill + 4 decode (12 units, 4 completions/iteration). If imbalanced (6 prefill, 3 decode), FCFS yields 3-3.5 completions instead of 4—a 12-25% loss despite sufficient capacity. Evicted prompts restart from prefill (stage $s = 0$), perpetuating the imbalance cycle.

Our Approach. We use fluid dynamics to characterize equilibrium and optimal M^* . We design threshold-based algorithms: WAIT (known outputs) and Nested WAIT (unknown outputs via on-the-fly classification). Validation on LMSYS-Chat-1M shows 12-25% improvement over vLLM/Sarathi on A100.

2. Model and Fluid Dynamics

2.1 System Model

Notation. Table 1 defines key notation.

Table 1: Key Notation

Symbol	Description
m	Number of prompt types
j	Type index, $j \in \{1, \dots, m\}$
λ_j	Arrival rate of type- j (Poisson)
l_j	Input length (tokens) after prefill
l'_j	Output length (tokens) in decode
s	Stage: $s = 0$ (prefill), $s \in \{1, \dots, l'_j\}$ (decode)
C	GPU memory capacity
M^*	Equilibrium memory
τ	Iteration time to process batch
d_0	Fixed overhead per iteration
d_1	Time cost per unit KV cache
Θ^*	Equilibrium throughput (decode tokens/time)

Prompts and Processing. Prompts of type j arrive via Poisson process with rate λ_j . Each type- j prompt requires one prefill iteration (consuming l_j memory units) followed by l'_j decode iterations (each adding 1 memory unit). A prompt at stage s occupies $l_j + s$ memory units.

Iteration Time Model. The iteration time τ captures two costs: fixed overhead (kernel launch, synchronization) and memory-dependent cost (KV cache loading). We estimate parameters d_0 and d_1 from GPU profiling data. For a batch with total KV cache M :

$$\tau = d_0 + d_1 \cdot M. \quad (2.1)$$

This linear model captures memory bandwidth cost: d_0 is fixed overhead (kernel launch, synchronization), d_1 reflects KV cache loading time. This model fits real GPU profiling data and enables data-driven parameter estimation from serving logs.

2.2 Fluid Equilibrium

We develop a fluid model to characterize the equilibrium state where arrivals match completions. This equilibrium quantifies maximum throughput Θ^* and reveals the load-balancing principle guiding our algorithms.

In equilibrium with n_j^* active prompts of type j , distributed evenly across $(l'_j + 1)$ stages, total memory equals:

$$M^* = \sum_{j=1}^m n_j^* \left(l_j + \frac{l'_j}{2} \right), \quad (2.2)$$

where $(l_j + l'_j/2)$ is the average memory per prompt across its lifecycle. The equilibrium condition (arrivals = completions) yields:

$$(d_0 + d_1 M^*) \lambda_j (l'_j + 1) = n_j^*, \quad \forall j. \quad (2.3)$$

Solving this system gives the equilibrium memory:

$$M^* = \frac{d_0 \sum_{j=1}^m \lambda_j (l'_j + 1) (l_j + l'_j/2)}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1) (l_j + l'_j/2)}. \quad (2.4)$$

The equilibrium throughput (decode tokens per unit time) is:

$$\Theta^* = \sum_{j=1}^m \lambda_j l'_j. \quad (2.5)$$

This equilibrium reveals a fundamental principle: fixed overhead d_0 means larger batches amortize cost, making throughput proportional to batch size. Memory is a resource to exploit, not just a constraint—the equilibrium shows how to maximize utilization while avoiding overflow. When the system operates near M^* , it achieves this balance.

3. WAIT Algorithm: Known Output Lengths

3.1 Motivation: Why FCFS Fails

As illustrated in Section 1, FCFS causes cascading evictions even when $C \geq M^*$. The root cause: FCFS admits prompts greedily, causing the system to drift from equilibrium. Once imbalanced, one type overflows, triggering evictions; freed memory fills with another type, which itself overflows—a vicious cycle. Memory sufficiency alone doesn't guarantee stability—the scheduling policy must maintain load balance.

3.2 WAIT Algorithm

The WAIT (Waiting for Accumulated Inference Threshold) algorithm prevents eviction cascades through threshold-based admission. For each type j , WAIT sets a threshold n_j derived from fluid dynamics. At time t , WAIT monitors inventory n_{j0}^t (prompts of type j waiting at prefill). Type j is batched only when $n_{j0}^t \geq n_j$.

Algorithm. When $n_{j0}^t \geq n_j$ for some type j :

- Select $\min\{n_j, n_{js}^t\}$ prompts of type j at each stage $s = 0, 1, \dots, l'_j$
- Process the batch; keep other prompts waiting (preserve their KV caches)

If no type meets the threshold, wait for more arrivals. We set thresholds proportionally: $n_j \propto \lambda_j / (l'_j + 1)$, aligning with equilibrium distribution.

Intuition. Thresholds decouple multi-type dynamics through algorithm design—not inherent problem simplicity—enabling tractable single-type analysis. This decoupling is the key to asymptotic optimality. Waiting until threshold ensures batch size approximates equilibrium, fully exploiting memory capacity while preventing overflow. The system approaches the fluid equilibrium, maintaining load balance that prevents cascading evictions.

Theorem 1 (Near-Optimality). *Under $C \geq M^*$, WAIT achieves throughput*

$$\mathbb{E}[\Theta^T] \geq \Theta^* - O\left(\frac{\log(mT)}{T}\right) \quad \text{as } T \rightarrow \infty. \quad (3.1)$$

4. Nested WAIT: Unknown Output Lengths

4.1 Challenge

In practice, output length l'_j is unknown at arrival—prediction is costly and imprecise. Traditional shortest-job-first methods degrade when output lengths are uncertain.

4.2 On-the-Fly Classification

Prompts self-classify as they decode. We group the m decode stages into $L \ll m$ segments. Short prompts complete within early segments and exit; long prompts naturally advance to later segments. No prediction needed—classification emerges from actual decoding.

For example, with 10 types having decode lengths 50, 100, ..., 500 tokens, we create 10 segments of 50 tokens each: segment 1 covers stages 1-50, segment 2 covers stages 51-100, etc. A prompt generating 75 tokens completes in segment 2 and exits, while a 300-token prompt advances through segments 1-6.

4.3 Nested WAIT Algorithm

For each segment $k = 1, \dots, L$, set threshold n_k . At each iteration:

- Check if $\sum_{j:\text{stages in segment } k} n_{j0}^t \geq n_k$
- If yes, batch all prompts with stages in segment k
- Short prompts exit after early segments; long prompts continue

We set thresholds n_k proportionally to arrival rates in each segment, with a safety buffer to hedge output length uncertainty.

Theorem 2 (Unknown Output Optimality). *Nested WAIT achieves near-optimal throughput without knowing l'_j at arrival:*

$$\mathbb{E}[\Theta^T] \geq \Theta^* - O\left(\frac{\log(LT)}{T}\right). \quad (4.1)$$

The segment design adapts to varying distributions and ensures stability when long decode lengths prevent guaranteeing prompts at every stage.

5. Experiments

5.1 Setup

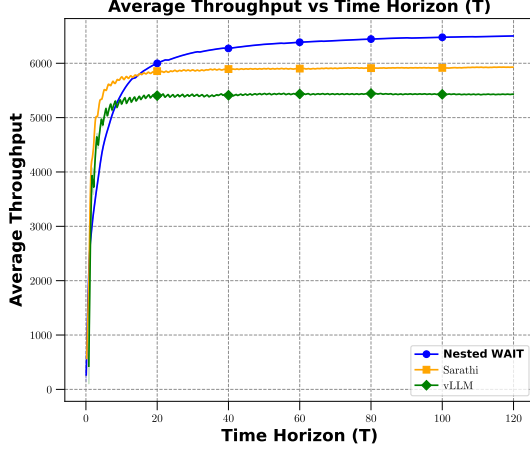
We evaluate on Microsoft Vidur simulator calibrated with NVIDIA A100 GPU profiling data. Baselines: vLLM [4] (prioritizes new arrivals) and Sarathi [5] (prioritizes ongoing prompts), both using FCFS scheduling with token limits.

Dataset. We use LMSYS-Chat-1M [6], randomly sampling 50K prompts from the 210K dataset (Vicuna and Chatbot Arena). Decode lengths range 1-500 tokens. We group these into 10 segments with thresholds adapted to arrival distribution, enabling on-the-fly classification where short prompts complete early without blocking.

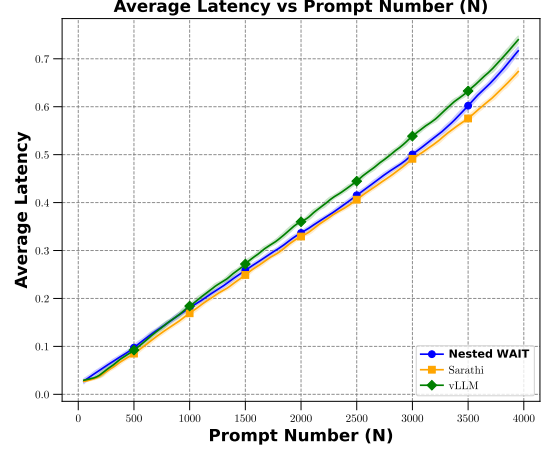
Parameters. Batch size $B = 148$, thresholds n_k proportional to segment arrival rates. GPU profiling yields $d_0 \approx 0.01\text{s}$ (fixed overhead), $d_1 \approx 10^{-5}\text{s}$ per token (memory bandwidth). We test at two arrival rates: 55 QPS (low demand) and 550 QPS (high demand).

5.2 Results

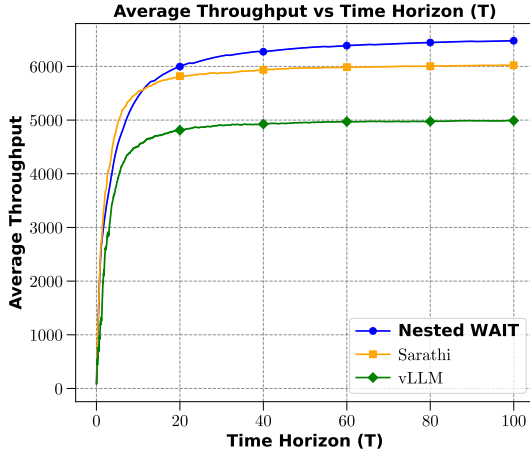
Figure 1 compares performance across both arrival rate regimes. At high load (550 QPS, top row), Nested WAIT achieves 12-25% throughput improvement over vLLM and Sarathi while maintaining competitive latency. The algorithm prevents cascading failures through controlled admission that keeps systems near fluid equilibrium. At low load (55 QPS, bottom row), Nested WAIT incurs slightly higher latency due to threshold waiting but eliminates eviction risk entirely, demonstrating the throughput-latency tradeoff inherent to controlled admission.



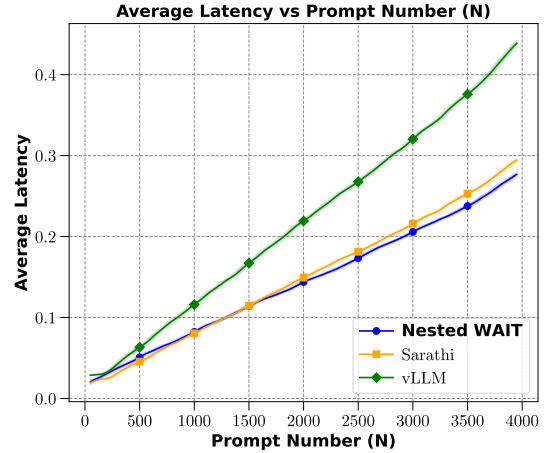
(a) Throughput (High QPS)



(b) Latency (High QPS)



(c) Throughput (Low QPS)



(d) Latency (Low QPS)

Figure 1: Performance on LMSYS-Chat-1M at 550 QPS (top) and 55 QPS (bottom). Nested WAIT improves throughput 12-25% while maintaining competitive latency.

6. Related Work and Conclusions

6.1 Related Work

Operations Research Perspective. Recent works [7–9] analyze LLM scheduling from online algorithms viewpoint. Jaillet et al. assume constant $\tau = 1$, missing memory bandwidth effects. Our $\tau = d_0 + d_1 M$ model reveals how larger batches increase throughput but risk overflow—the core trade-off that drives eviction prevention. Li et al. [10] study general batch processing; our work focuses on memory-centric eviction prevention.

System-Level LLM Inference. vLLM [4] and Sarathi [5] use FCFS-based batching but lack eviction prevention mechanisms. DistServe [11] and Splitwise [12] implement Prefill-Decode disaggregation; our model applies directly to decode servers where the linear iteration time model holds exactly.

Queueing Theory. Fluid models [13, 14] yield first-order approximations for stochastic systems. We adapt these techniques to LLM inference’s unique challenges: dynamic memory growth, eviction feedback, and multi-stage processing.

6.2 Conclusions

We develop fluid-guided threshold-based algorithms for LLM inference scheduling under memory constraints. We contribute: (1) We model memory constraints by capturing dynamic KV cache growth. (2) We design eviction-prevention algorithms (WAIT and Nested WAIT) that achieve asymptotic optimality. (3) We validate these algorithms on real-world data, achieving 12-25% throughput improvement.

Our data-driven framework bridges operations research theory and AI systems practice, providing actionable insights for LLM serving infrastructure. Performance metrics from real-world serving logs and GPU profiling inform scheduling decisions, demonstrating how OR methods—grounded in system data and analytics—optimize modern AI infrastructure.

Future Work. Extensions include multi-GPU settings, time-varying arrivals, and disaggregated memory systems.

References

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [2] OpenAI, “Gpt-4 technical report,” 2024.
- [3] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in nlp,” *arXiv preprint arXiv:1906.02243*, 2019.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- [5] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, “Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills,” *arXiv preprint arXiv:2308.16369*, 2023.
- [6] L. Zheng, W.-L. Chiang, Y. Sheng, T. Li, S. Zhuang, Z. Wu, Y. Zhuang, Z. Li, Z. Lin, E. P. Xing, *et al.*, “Lmsys-chat-1m: A large-scale real-world llm conversation dataset,” *arXiv preprint arXiv:2309.11998*, 2023.
- [7] P. Jaillet, J. Jiang, K. Mellou, M. Molinaro, C. Podimata, and Z. Zhou, “Online scheduling for llm inference with kv cache constraints,” 2025.
- [8] M. Wang, Y. Ye, and Z. Zhou, “Llm serving optimization with variable prefill and decode lengths,” *arXiv preprint arXiv:2508.06133*, 2025.
- [9] Z. Chen, Y. Ye, and Z. Zhou, “Adaptively robust llm inference optimization under prediction uncertainty,” *arXiv preprint arXiv:2508.14544*, 2025.
- [10] Y. Li, J. Dai, and T. Peng, “Throughput-optimal scheduling algorithms for llm inference and ai agents,” 2025.
- [11] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 193–210, 2024.
- [12] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” pp. 118–132, 2024.
- [13] A. Mandelbaum, W. A. Massey, and M. I. Reiman, “Strong approximations for markovian service networks,” *Queueing Systems*, vol. 30, no. 1, pp. 149–201, 1998.
- [14] C. Maglaras, “Discrete-review policies for scheduling stochastic networks: Trajectory tracking and fluid-scale asymptotic optimality,” *The Annals of Applied Probability*, vol. 10, no. 3, pp. 897–929, 2000.